NASA Space Explorer

Junsu Kim · November 5, 2026

1. Website URL

https://playlist0726-code.github.io/nasa-space-explorer/

GitHub repository: https://github.com/playlist0726-code/nasa-space-explorer

1B. LevelUps completed

All three LevelUps are included (25 possible extra-credit points):

- **APOD video entries (10 pts):** Video and iframe entries appear as clearly labeled gallery cards. Their detail view includes the explanation and a direct link to watch the video on NASA's page. Try January 1-9, 2025, and open the January 5 video.
- **Random space fact (10 pts):** A “Did You Know?” section above the gallery randomly selects a fact on each page load. The refresh button shows a different fact.
- **Hover zoom (5 pts):** Gallery images smoothly scale on hover. Reduced-motion preferences are respected.

2. Reflection: working with a real API

Working with NASA's APOD data showed me that connecting to a real API involves more than displaying an image URL. Each entry has a date, a title, an explanation, and a media type. The interface has to work with those differences and remain understandable while a request is loading or fails.

A specific challenge in this project was the API's recent migration to NASA Science. In the newer response, the url field can point to an article rather than an image. The explanation and credit fields can also contain HTML. The implementation uses hdurl for images and converts the descriptive HTML into safe, readable text. Video entries get an explicit watch link instead of a broken image.

Another challenge was fetching the full selected date range. NASA's newer feed defaults to 25 records, so the app splits longer searches into bounded requests, combines the results, and checks that returned dates belong to the requested range. It also reports unavailable days instead of silently pretending that every date was returned. Loading messages, retry controls, and date validation make the page usable when the data source is slow or unavailable.

AI supported the implementation and verification process. The most useful lesson was to compare the current documentation with a real response and then test the behavior in a browser. A request that succeeds

is only the first step; the result still needs to be interpreted correctly.

3. Interview plan: how I approach debugging

I would explain my process in five steps: reproduce the problem, narrow down where it occurs, inspect the evidence, make one focused correction, and verify the result with a regression check.

For this project, I would use the APOD image URL issue as an example. The symptom was that an API field named url did not necessarily identify an image. I inspected an actual NASA response and compared it with the current official APOD API source. That showed that the migrated API uses a post permalink for url and a separate hdurl for the photograph. The problem belonged to response handling rather than the gallery layout.

The correction was to prefer hdurl for image entries, handle videos separately, and use the NASA permalink for the source link. I also converted HTML descriptions to plain text so that the explanation and credit fields were displayed safely. To verify the correction, I tested actual image and video entries in the browser and added automated coverage using a fixture where url points to an article. This check helps prevent the same mistake from returning later.

I would close the interview answer by describing how I handle uncertainty: use the browser's Network and Console panels to check the request status and response, isolate one variable at a time, and test both the successful path and failure states before calling the issue resolved.

4. LinkedIn post draft

This week I built Space Explorer, a responsive web app that connects to NASA's Astronomy Picture of the Day archive.

You can pick a date range, browse space images, and open each discovery to read the full explanation. I also added video support, random space facts, and a subtle image zoom effect.

The most useful lesson came from working with a changing API. NASA's newer response can use url for an article and hdurl for the actual image. Checking real responses against the current documentation helped turn that difference into a concrete fix. I also practiced handling loading states, validating dates, and testing mobile layouts.

I used AI to support development and verification, and learned why every generated solution still needs to be checked against real data.

Explore the project: https://playlist0726-code.github.io/nasa-space-explorer/

#WebDevelopment #JavaScript #NASA #APIs #LearningInPublic